

« Chapter 15.1: More Tools to the Scala Toolbox (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 15 (https://plus.cs.aalto.fi/studio_2/k2022/w15/) / Chapter 15.2: Exceptions and attempts

Chapter 15.2: Exceptions and attempts

About this page:

Key questions: How to deal with special cases.

Topics: Learn about exceptions and two alternative ways to deal with code that can throw an exception.

What will I do? Reading and programming

Indicative difficulty level: Intermediate

Rough estimate of workload? 3-4 hours.

Point value: 50

Related projects: [ExceptionalCoffee](https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/ExceptionalCoffee/) (https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/ExceptionalCoffee/).

Exceptions

There are almost always a number of points in a program where the attempted action can, in one way or another, fail. A well-designed program should behave in a meaningful way even when something goes wrong. Our program should not "crash" except in extreme cases.

An *exception* is a "minor special situation" arising when executing a program, which jeopardizes its operation. Most programming languages understand the concept of an exception. Examples of actions leading to exceptions include:

- Trying to divide by zero: `1/0`
- Trying to read a file that does not exist or the program has no reading access rights to the file.
- Trying to find the largest number in an array/list whose size is zero:
`List[Int]().max`

- Trying to interpret a string as a number (`text.toInt`), but it turns out that the string is invalid for this purpose, e.g. “cat”. (See, for example, string “123” for which the operation is successful.)
- Trying to make an illegal type conversion, e.g., convert an `Auto` object into a `Student` object.

Error

Some programming languages, such as Java, distinguish between exceptions that can typically be recovered from the more serious problems. Errors in Java terminology are the more serious situations that almost certainly make it impossible to continue the program execution (i.e., the program crashes) and for which it is difficult to come up with a reasonable way to recover. Technically, however, they operate with the same mechanism as exceptions.

Examples of errors:

- Trying to create a new object, when the memory of the computer is full.
- Some of the program's class files have been destroyed or cannot be accessed.
- Trying to invoke a method that is implemented using the `???` method whereby a `NotImplementedError` occurs.

Where do we use exceptions?

Often, an exceptional situation cannot be treated reasonably right there (in the method) where the exceptional situation arises. For example, what should the `toInt` method do if its parameter is invalid? The general utility of such a method would decisively decrease if it would “decide the action itself”. Sometimes the method return value can be used (and is used) to describe special situations, but this solution has limitations.

The method below returns the ratio of the smallest value of an integer collection to its highest value.

```
def getRatio(b: Buffer[Int]) = {  
    b.max / b.min  
}
```

What do we do if the buffer is empty? Or if the smallest element is zero?

The exception handling is based on the following: it is possible to *throw an exception* (or *raise/cast an exception*), i.e., to forward the exception information to a part of the program that defines how the exceptional situation can be resolved. If a method could not throw an exception, the special situations should be treated within the method in which they arise, or the information from the situation should be conveyed in some less appropriate way (usually using the method return value).

Exceptions are described in Scala (and Java) as objects, which shouldn't be surprising. All exceptions are instances of the class `java.lang.Exception` either directly or indirectly. The exact type of the exception depends on what kind of problem or special case the exception tries to describe. Even if it is possible to create "general purpose" exceptions from the `Exception` class, by using this approach we would waste the opportunity to tell more about the situation. Normally, it is more meaningful to create a specialized object in one of the subclasses of this "basic class". This allows us to use the type and content of the exception for more accurate communication.

```
if (didNotWork) {
  // The exception has always a textual message about what went wrong
  // The object created directly from the Exception class does not have other possibilities to transfer the information

  val actionXFailed = new Exception("The action X failed.")

  // Doing something with the exception object...
}

if (stillFails) {
  // A custom, or ready made but more appropriate type of exception allows for more accurate
  // delivery of information by the exception type and values inside the exception object.

  val moreClearly = new ElevatorStuckException("Elevator door stuck", elevatorNumber, floor)
}
```

In practice, exceptions are almost always created when we want to inform the caller of the method to be executed that the execution of the method failed for some reason, that is, we raise an exception. The caller method can then decide on how to deal with it. The exceptions thus form a communication channel from the called method to the caller.

Throwing exceptions

"Nothing travels faster than the speed of light, with the possible exception of bad news, which obeys its own special laws."
-Douglas Adams, *Mostly Harmless*

Scala and Java have a special statement `throw` for throwing an exception. One of the exception objects is given to the statement. **The `throw` statement interrupts the normal execution of the method immediately.** Execution will be continued from the first place found in the program which contains a procedure for dealing with such exceptions. Usually, such an exception handling structure cannot be found in the same method with the `throw` statement. In this case, the exception is thrown out of the method, i.e. the execution of the method is completely interrupted and the exception handler will be looked for in the calling method. We will return to how such an exception handling structure is written in just a moment.

```
if (didNotWork) {
    // Someone gave us an impossible task !!
    // I shall throw an exception so they can decide
    // for themselves what should be done

    throw new Exception("Action X failed");

    // The above exception is created and thrown at the same time.
    // (These two things are very often indeed done together.)

    println("This message cannot be seen")

    // These lines will not be executed if the exception was thrown.
    // Instead, execution jumps to a place in the program which includes
    // exception handling commands that match the type of exception.
}
```

Different types of exceptions

As mentioned earlier, a variety of classes describing exceptional cases are inherited from the `Exception` class. This allows the separation of the different exceptions, if several different exceptional circumstances could occur. In addition, the exception objects may differ from each other in other ways than just in their names.

Most Java libraries - including, of course, Java's basic API - include exception classes related to the library's themes. Some ready-made exception classes are usable as such. For example, `IllegalArgumentException` describes the situation where the method was given a "non-contractual" parameter. This, such as most of the other existing exception classes, can be found in Java's class libraries. The Scala API uses almost the same set of exceptions as defined by the Java API.

However, the exceptions in the libraries are not specifically designed for the theme or implementation that we are building. When a tailor-made exception class is not available, it is easy to inherit it from the `Exception` class (or one of its subclasses). Even defining a subclass of the `Exception` class that does not define any new functionality can well be justified. The mere fact that this is an `Exception` subclass which is intended to be used in certain types of exceptional circumstances can be useful. The additional information is then conveyed only in the exception class type.

Your own exception type

Let us create a constructor for the class `Student` that can be given the `String` parameter "Teemu Teekkari 12345" and which then initializes the instance variables of the object using this string.

```
class Student(information: String) {
    private val parts = information.split(" ")
    // String class has a split method that divides the string
    // by using the given parameter (here space) as a separator

    val firstName = parts(0)
    val surname = parts(1)
    val studentNumber = parts(2)
}
```

This example is somewhat artificial, but it might be justified, for example, if we knew that Student objects should be created based on such text files which include student information stored in lines. Though, such things should not be done in the constructor, but for simplicity, let us look at this anyway.

```
val student = new Student("Teemu Teekkari 12345")
println(student.studentNumber)
```

What happens if the constructor receives just "cat" or "Teemu Teekkari" as a parameter? The constructor does not even have a return value that could be used for communication. It would be very strange if we were forced to create a Student object. Thankfully, the exception will interrupt the constructor's operation and no object will be created.

Extractors in the assignment statement

The extractor which we learned in the context of the match statement can also create a set of variables at once.

```
class Student(information: String) {
  val Array(firstName, surname, studentNumber) = information.split(" ")
}
```

information.split returns an array.
Array(firstName, surname, studentNumber) is syntactic sugar for the Array class extractor ([http://www.scala-lang.org/api/current/index.html#scala.Array\\$@unapplySeq\[T\]\(x:Array\[T\]\):Option\[IndexedSeq\[T\]\]](http://www.scala-lang.org/api/current/index.html#scala.Array$@unapplySeq[T](x:Array[T]):Option[IndexedSeq[T]])) which fetches the array values and assigns them to the given variables. Instead of an array, this results in 3 immutable (val) variables: firstName, surname and studentNumber.

If the Student class constructor were called with the parameter value "Teemu Teekkari", an `ArrayIndexOutOfBoundsException` exception would be created when storing the student number, due to trying to find an array element from a place outside its index range. This is, however, not very useful information for the class calling the constructor, because to be able to handle the exception it should know how the constructor has been implemented.

Let us rather create an exception type that is suitable for the purpose, which can be used to carry this particular type of exception information.

```
case class InvalidStudentData(description: String, errorData: String)
  extends java.lang.Exception(description)
```

extends java.lang.Exception(kuvas)

The class is inherited from the general Exception class and the constructor is given an error message.

errorData

We store (in addition to the actual description message) information about which data string caused the error in each exception object of this type.

case class

Exception handling will use the match structure. The case class offers just the right tools here.

How should the Student class be changed so that we can apply our new exception?

```
class Student(information: String) {
  val Array(firstName, surname, studentNumber) = {

    val parts = information.split(" ")

    // Let's first see whether the data seems to be right. If it is not, an exception is thrown.
    if (parts.length != 3) {

      // Since there is no special structure in the same method that would handle the exception,
      // the exception is thrown out to the calling method and the constructor's execution ends
      // at the throw statement. (There will be no new Student object.)

      throw new InvalidStudentData(
        "Student data did not have exactly three words.", information)
    }

    // The parts variable now certainly contains those three parts that can be assigned to firstName,
    // surname and studentNumber. These assignments can now be carried out
    // safely knowing that there is enough data. (Of course, there is nothing here to guarantee that
    // the given data would be semantically sensible.).

    parts
  }
}
```

Catching exceptions

A method that calls another method that might throw an exception can declare that it handles (catches) the exceptions itself (or at least certain types of exceptions).

This is implemented with the try-catch statement: "I try to do these things, but if something goes wrong, I catch an exception and look at what I'll do with it." If an exception is thrown in a method call within a try block, skip the remainder of the block and move to the catch block. If the execution of a try block succeeds smoothly, no catch block is executed.

Catch blocks can be used for several different types of exceptions. Often one is enough. When the end of one of the blocks is reached, the execution of the program continues as usual from the statements after the try-catch sentence. Only one block is therefore executed.

```
try {
  // Code which at least in one point calls
  // a method that may throw an exception.
} catch {
  case SomeException(text) => // The code that responds to exception SomeException.
                              // the variable text will provide an exception description text

  case e:SomeOtherException => // The code that responds
                              // to this second exception type.
                              // The variable e is a reference to an exception object

  case _ =>                  // Error capturing without an error type is a
                              // very very bad idea, as it just hides
                              // still unknown errors. If your program would divide by zero,
                              // you would not notice it
}
```

As in the case class-match statement, cases will also be processed in exception handling from top to bottom one at a time until the right block is found. For this reason, cases should be organized from the most specific one to the most general one, to avoid that the general case manages all exceptions.

The student example continues...

Let us try to create a new student object and print its student number. In Scala try is a statement that can, if desired, return a value. In our example, the value of try is stored in the variable stud.

```
val student: Option[Student] =

  // A Student class constructor can throw an exception and right here
  // is a good place to deal with it. Place the statements in the try block. If
  // the constructor casts an exception, no print statement is executed, but
  // we jump to the catch section.

  try {
    val stud = new Student("Teemu Teekkari")
    println(stud.studentNumber)
    Some(stud)
  } catch {

    // The following block handles exceptions of the class
    // InvalidStudentData (including possible subclass instances).

    case InvalidStudentData(message, data) => {
      println(message)
      println(s"Invalid data line was $data.")
      None
    }
  }
}
```

When executed, the program prints the following:

```
Student data did not have exactly three words.
Invalid data line was Teemu Teekkari.
```

printStackTrace:

An exception object stores the **Call stack** when the exception was created.

The Exception class method `printStackTrace` prints a *stack trace* that shows which method calls have caused an exceptional situation. You've certainly seen such a printout when some of your programs have crashed as a result of an error.

The `printStackTrace` and `getMessage` methods, as well as other methods defined for that exception type, can help you find errors in the program. One of the strengths of exceptions is precisely that they can easily convey descriptions and additional information about problem situations.

Moving the exception forward

If the exception is not captured in one of the methods, it will proceed in the call stack to the following method. All the methods that do not catch the exception will stop. If an exception is thrown out of the entire program, the entire program stops and the virtual machine prints out the description of the exception using the `printStackTrace` method. However, this should never happen.

Warning of the possibility of exception

In Scala (as opposed to Java), there is no need to warn the user of the method that the method may throw an exception. In Java, the exceptions are divided into exceptions that are subject to checking and those that can be thrown freely. Exceptions which are checked must have an exception handler, while for free exceptions, such as, for example, `ArithmeticException` reporting of divide by zero, you do not need to write an exception handler. Exceptions which must be checked are one of Java's features, which strongly split opinions. Scala deals with all exceptions as they would be free to cast.

```
// In Java, the exceptions to be checked are marked with a warning "throws"
// and they are also listed in the method documentation, on the so-called javadoc pages.
// Since the Java code is called from Scala, it is good to recognize the notation.1

public int exampleMethod() throws ExceptionType1, ExceptionType2 {
    // THIS IS JAVA CODE !!!
}
```

Chaining exceptions

In the first version of our `Student` class that used the `Array` class extractor, the number of parts in the string was not checked. In this case, the `Array` class extractor would have thrown `MatchError`. Despite its name, this is not a serious error, but we could have captured this exception and thrown a new exception on the basis of what we have captured.

If desired, we can tell that one exception was caused by another exception. In this case, the `initCause`

([http://docs.oracle.com/javase/7/docs/api/java/lang/Throwable.html#initCause\(java.lang.Throwable\)](http://docs.oracle.com/javase/7/docs/api/java/lang/Throwable.html#initCause(java.lang.Throwable)))

method is used. It takes the original exception as a parameter. This is sometimes very useful when examining the root causes of the exceptions.

```
class Student(information: String) {
  val Array(firstname, surname, studentNumber) = try {
    information.split(" ")
  } catch {

    case e: MatchError =>
      val newException = new InvalidStudentData(
        "Student data did not have exactly three words.",
        information)

      newException.initCause(e)
      throw newException
  }
}
```

try-catch-finally

In addition to the catch sections, a finally block can be written after the try block. Finally block is **always** executed at the exit of the try block - even when any possible catch blocks have been executed. Finally will thus be executed, even when the execution exits the try block with a free thrown exception or consciously in the middle of the block by using the return statement.

Finally block is performed after the try exception has received the value it will return. Therefore, it is often a safe place to perform some "cleaning operations", as any run-time errors cannot prevent executing these lines of code. When the finally block has been completed, the execution of the code continues as it would have done without the finally block.

Option and Exceptions

In some situations where the Java code would have to cast an exception, Scala may have the special situation handled using the Option class. In a situation where a method cannot return a value (because something went wrong), it would return None, otherwise Some(...).

This allows in the normal case returning a value and in special cases returning None, which informs that something exceptional happened. But how could we differentiate between different error situations? In our previous student example, the Option object was assembled using a try-catch structure. But could we do this in an easier way?

Try (with big T)

Try (with Big T) is a data type that is in many ways a close relative of the Option class. Like Option, Try is also an abstract class with two subclasses: one for Success, another for Failure. These behave in a very similar way as the classes of Some and None with functions like `map`.

Try's constructor (or, in fact, the companion object's `apply` method) takes as its only parameter an expression that can produce either a value or cast an exception:

```
import scala.util.{Try, Success, Failure}

//...
var couldBeNumber = Try {
  line = readLine("give a number!")
  line.toInt
}

couldBeNumber.foreach(println)
```

If there is no exception, the above will give `Success`, which can be treated in a similar way as a `Some` object.

If there is an exception, the result is `Failure(java.lang.NumberFormatException(message))`. `Failure` is therefore a wrapper class that contains the exception that was raised.

The last line in the example, `couldBeNumber.foreach(println)` demonstrates similarity with `Option` class. Because `Failure` behaves like `None` with higher-order functions, this statement prints the content of the `couldBeNumber` variable only if it is in the `Success` wrapper.

Should `Try`'s content, or whether it is a `Success` or `Failure` be investigated, the `match` structure is – once again – quite convenient:

```
couldBeNumber match {
  case Success(number) => println(number)
  case Failure(e) => println("there was an exception" + e)
}
```

In addition, `Try` has `isSuccess` and `isFailure` methods. Its `get` method either returns a successful value or throws an exception (in case of `Failure`). The structure corresponding to the `catch` statement is called `recover`, which takes as a parameter a so-called partial function, which is a function defined only for certain parameter values and types.

```
c = Try(a / b) recover {case e: ArithmeticException => 42}

// If we divide in our example with zero, the result is Failure, which recover handles, and spits out Success(42).
```

Let us now work with one exercise, where you can work with exceptions.

3. Exceptional Coffee (50p)

Exceptions often result from working with IO devices, concurrency or input that comes from outside of our program. In this exercise we have to use a fictional Coffee machine that can throw exceptions.

There are two methods that need to be implemented in the class `Cafe`: **`makeOrderTryCatch`** and **`makeOrderTry`**. Both receive an amount of brewing attempts and should return a buffer of `Coffee` instances created by the machine. **Use a `try-catch` in one and a `Try` in the other.**

How the methods should work

- Try to make the asked amount of coffee using coffeemaker. If an exception is thrown, the method returns one less coffee than asked amount, so you don't have to make

a new coffee if making coffee fails.

- The Coffeemaker can throw exceptions if it runs out coffee beans or becomes dirty. If making coffee throws **NoBeansException** or **MachineDirtyException**, fix problem using corresponding methods in the **Cafe** class.
- You should not handle other exceptions. Notably, if the machine breaks, it throws a different exception which in case of try-catch should not be caught and when using Try should be rethrown when examining the Try.
- Implement makeOrderTryCatch using a try-catch structure
- Implement makeOrderTry using Try and then a match-case with Success and Failure classes.
- Don't change order of the methods. Write all code inside methods.

Caveat

As the machine requires maintenance when dirty or out of beans we can't use truly functional approaches that the Try-variant normally offers. If the brews were independent from each other, We could use a more functional approach.

Submission

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 2 (Exceptional Coffee)

This course has been archived (Thursday, 1 June 2023, 12:00).

Select your files for grading

 **Cafe.scala** Show anyway

Choose File No file chosen

Submit

Summary

- We learned about exceptions, exception objects, and throwing and catching exceptions.
- As an alternative, more functional way of dealing with exceptions, we discussed the class Try

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don't have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

Show anyway

120

"I feel that I have understood the most important things in this chapter." (*) Pakollinen

- ☒ fully agree
- ☐ somewhat agree
- ☐ somewhat disagree
- ☐ fully disagree
- ☐ I'm unable to answer or don't want to comment.

The Try method with match-case structure would require me to HANDLE the case when MachineBrokenException(text) happens. Which is CLEARLY STATED NOT TO DO in the TASK.

Submit an update